

Week 1 Code Lab

Environment Setup and Your First Agent/Problem Skeleton

CMPSC 480 - Student Question Deck

Assignment overview

Build a verified Python development environment and a reusable agent/problem abstraction for a small gridworld.

This is a tooling and representation lab. You will not implement a search algorithm yet.

Expected effort

2-3 hours

Points

10 total

Submission

Git repository archive or
repository link

Learning goals

- ▶ Verify a reproducible Python, NumPy, Jupyter, and Git environment.
- ▶ Map a formal search problem onto a concrete Python interface.
- ▶ Represent states as immutable, hashable values.
- ▶ Define legal actions, state transitions, a goal test, and step costs.
- ▶ Use validation, type hints, docstrings, and unit tests from the start.
- ▶ Create abstractions that can be extended in Week 2 without redesign.

Required repository structure

Submit this structure

```
week01-agent-skeleton/  
  README.md  
  verify_environment.py  
  src/  
    __init__.py  
    agent.py  
    problem.py  
  tests/  
    test_problem.py
```

- ▶ The repository must contain at least one meaningful Git commit.
- ▶ The README must state the Python version and exact test command.
- ▶ Do not commit virtual environments, cache folders, or credentials.

Checklist - Environment and structure

1. **Environment report:** `verify_environment.py` reports Python, NumPy, Jupyter, and Git status.
2. **Version enforcement:** the script exits unsuccessfully when Python is older than 3.10 or a required package is unavailable.
3. **Git evidence:** the script confirms that it runs inside a Git repository with at least one commit.
4. **State contract:** every grid state is immutable, hashable, and comparable by value.

Checklist - Behavior and quality

5. **Problem interface:** implement `initial_state()`, `actions(state)`, `result(state, action)`, `goal_test(state)`, and `step_cost(state, action, next_state)`.
6. **Defensive behavior:** reject malformed states and illegal actions with precise exceptions; never mutate a caller-provided state.
7. **Agent interface:** define an extensible `Agent` base class with an `act(percept)` contract.
8. **Verification and documentation:** provide type hints, public-method docstrings, and at least five focused unit tests.

The toy domain

Gridworld

Use a rectangular grid with:

- ▶ integer coordinates (*row*, *column*),
- ▶ four actions: up, down, left, right,
- ▶ optional blocked cells,
- ▶ one initial cell and one goal cell,
- ▶ unit cost for every legal move.

Required conventions

- ▶ The upper-left cell is (0, 0).
- ▶ Rows increase downward.
- ▶ Columns increase to the right.
- ▶ `actions(state)` returns an empty list when no move is legal.
- ▶ The Week 1 code contains no search loop.

Task 1 - Environment verification (2 points)

Create `verify_environment.py`. A successful run must:

1. print the complete Python version;
2. reject Python versions earlier than 3.10;
3. print installed NumPy and Jupyter versions;
4. confirm that the current directory is inside a Git work tree;
5. report a positive commit count; and
6. exit with status code zero only when every check passes.

Key idea. The output must be machine-checkable: use stable labels and return a nonzero process status for a failed requirement.

Task 2 - Problem abstraction (5 points)

Implement a concrete `GridProblem` that owns the domain configuration and exposes the required search-problem operations.

- ▶ Validate dimensions, initial state, goal state, and blocked cells.
- ▶ Return legal actions in a documented deterministic order.
- ▶ Return a new immutable state from `result`.
- ▶ Reject an action that is not legal in the supplied state.
- ▶ Make the goal test return a real Boolean value.
- ▶ Return a positive numeric step cost only for a legal transition.

Task 3 - Agent abstraction (1 point)

Create an Agent base class with this public contract:

`act(percept)`

Accept the agent's current percept and return the next action selected by a concrete subclass.

- ▶ The base implementation must not pretend to choose an action.
- ▶ Document the expected percept and action types.
- ▶ Design the interface so Week 2 search agents can subclass it.
- ▶ Do not implement BFS, DFS, UCS, Greedy Best-First Search, or A-star.

Task 4 - Contract tests and repository quality (2 points)

Provide at least five focused tests. Your suite must include:

- ▶ a known legal transition;
- ▶ a corner or blocked-cell action set;
- ▶ an illegal-action exception;
- ▶ a goal-test success and failure;
- ▶ evidence that a state can be used as a set member or dictionary key;
- ▶ a domain with no legal actions;
- ▶ at least one malformed-input rejection; and
- ▶ a reproducible README with setup and test commands, plus a clean, meaningful Git commit.

You may combine closely related assertions in one test, but each failure must remain easy to diagnose.

Starter code - state and action types

```
from __future__ import annotations

from enum import Enum
from typing import TypeAlias

State: TypeAlias = tuple[int, int]

class Action(Enum):
    UP = (-1, 0)
    DOWN = (1, 0)
    LEFT = (0, -1)
    RIGHT = (0, 1)
```

Starter code - problem interface

```
class GridProblem:
    def initial_state(self) -> State:
        raise NotImplementedError

    def actions(self, state: State) -> list[Action]:
        raise NotImplementedError

    def result(self, state: State, action: Action) -> State:
        raise NotImplementedError

    def goal_test(self, state: State) -> bool:
        raise NotImplementedError

    def step_cost(
        self, state: State, action: Action, next_state: State
    ) -> float:
        raise NotImplementedError
```

Starter code - agent interface

```
from abc import ABC, abstractmethod
from typing import Generic, TypeVar

PerceptT = TypeVar("PerceptT")
ActionT = TypeVar("ActionT")

class Agent(ABC, Generic[PerceptT, ActionT]):
    @abstractmethod
    def act(self, percept: PerceptT) -> ActionT:
        """Return the next action for the supplied percept."""
        raise NotImplementedError
```

Edge cases and defensive-programming rules

- ▶ A state outside the grid is malformed even if its tuple shape is valid.
- ▶ A blocked cell is not a valid current, initial, goal, or next state.
- ▶ `result` must reject an action not returned by `actions(state)`.
- ▶ `goal_test` must not silently accept malformed input.
- ▶ A one-cell grid may have an empty action list and may begin at the goal.
- ▶ State values must remain safe as dictionary keys after every operation.

Submission instructions

1. Run `python verify_environment.py` from the repository root.
2. Run the complete test suite using the command documented in README.
3. Confirm that generated caches and virtual environments are ignored.
4. Commit all required source and test files with a meaningful message.
5. Submit the repository link or archive through the course system.

Do not submit

API keys, access tokens, editor settings, virtual environments, or code for any search algorithm.

Grading rubric - Correctness (8 points)

Requirement	Points	Evidence
Environment verification	2	Versions, Git status, commit count, and meaningful failure status
Immutable state model	1	Hashable value semantics with no caller-visible mutation
Legal actions and transitions	2	Correct boundaries, blocked cells, and deterministic actions
Goal test and step cost	1	Correct Boolean result and legal unit-cost transition
Defensive validation	1	Precise rejection of malformed states and illegal actions
Agent abstraction	1	Extensible, documented act contract

Rubric - Verification and quality (2 points)

Requirement	Points	Evidence
Focused unit tests	1	Required normal, boundary, illegal, no-action, hashability, and malformed cases
Documentation and repository quality	1	Type hints, docstrings, README, clean structure, and meaningful commit
Total	10	

Reflection prompt

In 150-250 words:

1. Map the formal search-problem components - initial state, actions, transition model, goal test, and path cost - onto your class design.
2. Identify one ambiguity you resolved while defining the gridworld contract.
3. Explain why your state representation will work with an explored set in next week's graph-search implementation.

Submit the response in `README.md`.